A Controlled Study of Planner-to-Executor Contract Design and Runtime Validation for Embodied-Agent Execution in Isaac Sim

Anonymous Authors

***Abstract*—We present a controlled systems study of embodied-agent execution in Isaac Sim that isolates planner-to-executor contract design and lightweight runtime validation. Across navigation and tabletop manipulation, we compare an under-specified direct-action contract (P0), a typed tool-call contract (P1), and the same typed contract with a brief self-check (P2), under bare dispatch (R0) or validation with a single retry (R1). In the fixed-runtime action-interface ablation, P0/R0 fails on all four evaluated tasks, whereas P1/R0 and P2/R0 succeed on all four and eliminate invalid actions. In the fixed-contract runtime-only ablation, validation plus one retry raises success from 2/4 to 4/4 by converting invalid-first-action runs into successful recoveries. Across the broader evaluation slices, the qualitative ordering stays stable: P0/R0 is weakest, P0/R1 recovers, P1 and P2 remain strong, and P2 is associated with lower planner/tool overhead than P1 in the covered comparisons. These results show that explicit action interfaces and lightweight runtime checks are meaningful systems levers for embodied-agent execution in the tested Isaac Sim setting.**

I. Introduction

Embodied-agent systems often fail at the boundary between planning and execution. A planner can identify the right task-level intention yet still emit an action name or argument structure that the executor cannot dispatch. Such failures expose not only planning errors but also the planner-to-executor contract, the action interface that mediates the handoff, and the runtime validation policy that decides whether a bad first emission should terminate the run or be repaired.

This execution boundary often remains implicit in embodied-agent studies, where broader evaluations and stronger end-to-end models dominate the narrative [1]–[5]. Those studies are valuable, but they entangle execution-stack design with backend scale and benchmark breadth. We therefore ask a narrower robotics-systems question: when the simulator setting and task families are held fixed, how much do planner-to-executor contract design and lightweight runtime validation affect invalid actions, recovery, success, and planner/tool overhead?

We study two task families in Isaac Sim: language-conditioned navigation and tabletop manipulation. The matrix compares three contract variants: an under-specified direct-action contract (P0), a typed tool-call contract (P1), and the same typed contract with a brief self-check (P2). It also compares two runtime variants: bare dispatch (R0) and validation with a single retry (R1). These variants are implemented through different planner instructions, but the manipulated system variables are the action interface presented to the executor and the runtime validation policy applied before dispatch.

The findings are stable across the covered slices. P0/R0 is consistently the weakest condition; adding runtime validation to that weak contract recovers fragile invalid-first-action failures; explicit contracts P1 and P2 eliminate the invalid-action failure mode in the fixed-R0 slice; and P2 is associated with lower planner/tool overhead than P1 in the covered comparisons. Harder tasks increase planner and tool activity without reversing the ordering. The paper is limited to this controlled simulator study and draws bounded execution-design lessons rather than new-model or benchmark claims.

This paper makes three contributions:

1) It presents a controlled Isaac Sim study that isolates two execution-design axes, planner-to-executor contract design and lightweight runtime validation, across navigation and tabletop manipulation tasks.
2) It provides evidence that typed tool-call contracts reduce invalid actions relative to an under-specified direct-action contract, while runtime validation with a single retry converts recoverable invalid-first actions into successful executions in both covered task families.
3) It extracts a compact architectural lesson from the tested setup: contract design, runtime validation, and a brief planner-side self-check are separate systems levers that shape reliability and planner/tool overhead without changing the underlying task matrix.

Fig. 1 summarizes the evaluated boundary: a task instruction reaches the planner, the planner emits an action request under one of the three contract designs, the runtime applies either bare dispatch or validate-and-retry, and the executor records the outcome metrics used throughout the paper. The same figure also carries the compact failure-repair trace that anchors the recovery discussion.

II. Related Work

A. Embodied and LLM-Based Robot Planning

Recent embodied and language-conditioned robot-planning work studies how high-level language intent is translated into executable robot behavior in simulation or on robots, including closed-loop language feedback, embodied multimodal models, vision-language-action policies, model-based language grounding, and systems-first integration pipelines such as Inner Monologue [1], [2], [6]–[8]. This also includes language-conditioned skill selection, code-generating control,

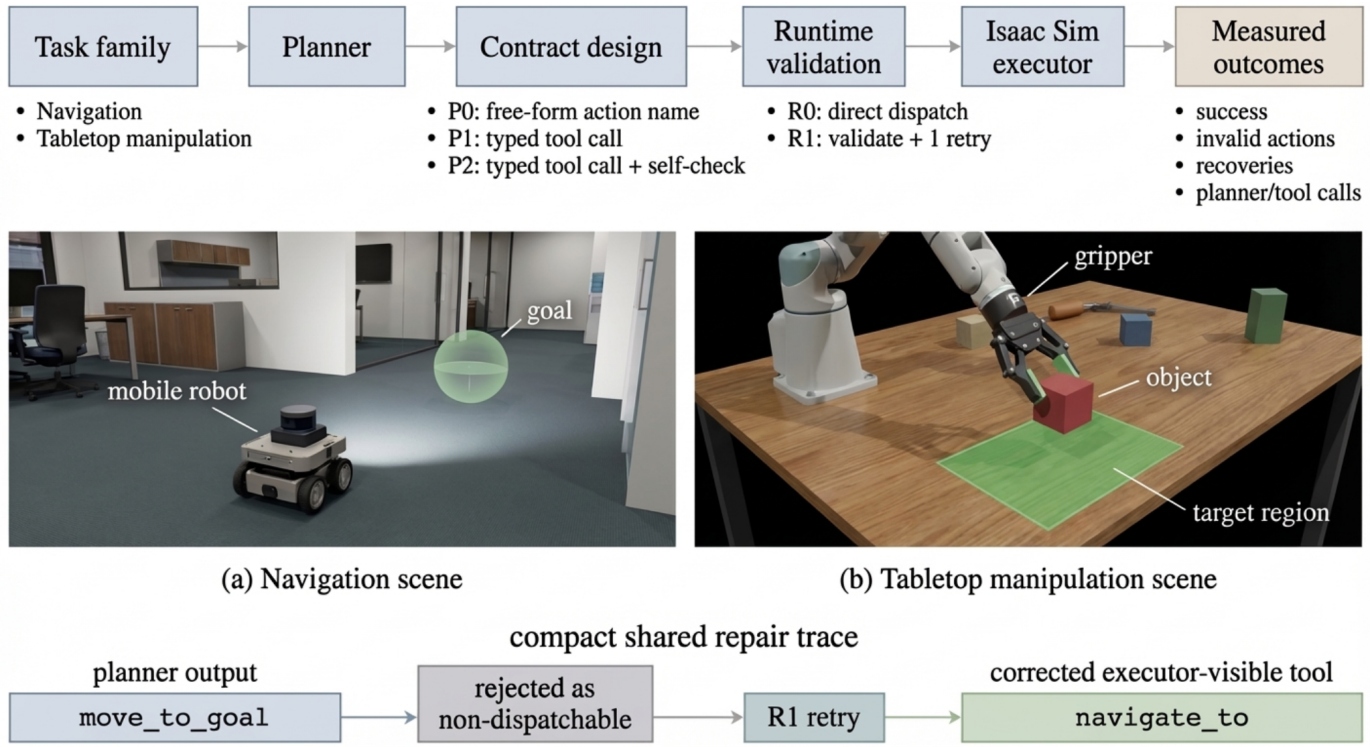

Fig. 1. System overview of the evaluated planner-to-executor boundary. A task is mapped by the planner into one of three contract variants, then passed through bare dispatch `R0` or validate-and-retry `R1` before the Isaac Sim executor. Insets show representative navigation and tabletop task scenes together with compact failure-repair traces in which non-dispatchable verbs are rejected and corrected into executor-visible tools.

and program-like robot planning directions exemplified by SayCan, Code as Policies, and ProgPrompt [9]–[11]. That literature makes planner quality visible, but it often leaves less room to isolate what happens at the boundary between planner output and dispatch. Our paper is narrower: it holds the Isaac Sim setting fixed and studies how planner-to-executor contract design changes invalid actions, recovery, success, and planner/tool overhead.

B. Execution Contracts, Runtime Validation, and Robotics Middleware

A second line of work addresses execution architecture rather than only planner quality. Relevant threads include ROS 2 middleware and action semantics [12], [13], planning frameworks such as PlanSys2 [14], behavior-tree execution stacks and libraries such as Behavior Trees in Robotics and AI and BehaviorTree.CPP [15], [16], and task-and-motion planning interfaces such as PDDLStream and the broader TAMP literature [17], [18]. The shared intuition is that planner outputs should not be treated as executable by default; they need an explicit interface to the execution stack. The present study is closest to that execution-boundary view. Its runtime layer is intentionally lightweight: R1 validates an emitted action or tool call against the active contract and allows one repair attempt. The contribution is therefore evidence about a small validation-and-retry mechanism in a controlled simulator setting, not a claim of a complete middleware stack or end-to-end safety case.

C. Controlled Design Studies and Broader Evaluations

Methodologically, the paper favors a small controlled matrix over benchmark breadth. Broader embodied-agent evaluations such as EmbodiedBench, IS-Bench, and the Isaac Sim-based Mind and Motion Aligned benchmark are valuable because they expose long-horizon execution difficulty, safety failures, or integrated planning-and-control behavior across larger task sets or evaluation matrices [3]–[5]. By varying only the action interface and runtime validation policy within a fixed Isaac Sim setting, we instead use a narrower study design to attribute differences in invalid actions, recovery, success, and planner/tool overhead to explicit execution-design choices. This complements broader embodied-agent evaluations rather than competing with them.

III. Study Design and Experimental Setup

A. Controlled Isaac Sim Setting

The study uses a common Isaac Sim execution architecture across navigation and tabletop manipulation [19]. The navigation family contains language-conditioned mobile-goal tasks in indoor scenes. The manipulation family contains tabletop tasks that require object identification, grasping, transfer, and placement. The main comparison uses a shared easy slice plus one harder slice per family, letting us test whether additional task burden changes the qualitative ordering or mainly amplifies planner/tool overhead.

All conditions share the same high-level handoff:

1) The planner receives the current task state and the exposed tool set.
2) The planner emits an action request.
3) Under R0, the request is dispatched as issued; under R1, the runtime validates it against the active contract and allows one repair attempt on failure.
4) The executor runs only requests that satisfy the active dispatch interface.

The dispatch vocabulary is family-specific and explicit. Navigation exposes state getters and `navigate_to`; manipulation exposes state getters and `scripted_pick_place_step`. The study therefore asks whether the planner can hand the executor a dispatchable request under a specified interface, not whether it can merely describe the task in natural language.

B. Contract and Runtime Variants

The controlled matrix is summarized in Table I. P0 uses an under-specified direct-action contract, allowing the planner to emit a task-level next action without binding it to the executor's tool namespace. P1 uses a typed tool-call contract, requiring the planner to choose one exposed tool and provide dispatchable arguments. P2 keeps the same typed tool-call contract and adds a brief planner-side self-check before emission.

R0 is a bare-dispatch runtime: it does not perform pre-dispatch validation or offer a repair path. R1 is a validate-and-retry runtime: it checks the emitted request against the active contract and, on failure, returns the validation error to the planner and allows one retry. Contract and runtime therefore play distinct roles. Contract design can prevent invalid actions at the source, while runtime validation can recover some contract violations that still slip through.

C. Implementation Snapshot

The frozen Block A evidence uses the repo-local `mock_block_a` planner backend rather than a hosted foundation-model API. In the archived run summaries, this appears as `planner_backend=mock_block_a` with `model_variant=null`; the implementation is the local file `src/isaacsim_agent/planner/mock.py`. The planner emits one deterministic JSON action per call, so there are no temperature, top-p, or stochastic decoding settings to tune in this frozen study.

P0 realizes a direct-action JSON contract, P1 realizes typed tool JSON, and P2 keeps the same typed tool JSON while adding a planner-side `self_check` field. R0 performs bare dispatch, while R1 validates tool name and arguments against the active task-specific namespace and allows one repair attempt before termination. Navigation exposes `get_robot_state`, `get_goal_state`, and `navigate_to`. Manipulation exposes `get_gripper_state`, `get_object_state`, `get_target_state`, and `scripted_pick_place_step`. The archived task configs, trajectories, and processed summaries record the selected backend realization, contract and runtime variants, termination reason, and Isaac runtime metadata such as `backend_variant` and `kit_version`=106.5.0, so the reported slices remain traceable to fixed local artifacts.

D. Metrics and Evaluation Protocol

Success rate is the primary task outcome. An invalid action is an emitted action or tool call that violates the active contract and therefore cannot be dispatched as issued. A recovery is a run that contains an invalid action but still ends successfully after runtime handling. Retries capture the extra repair work used in R1. Planner calls and tool calls together define planner/tool overhead, which is the paper's efficiency proxy.

The paper draws its quantitative comparisons from the frozen main factorial results plus targeted runtime-only, action-interface ablation, and harder-task summaries. The final closure summary provides the canonical aggregate view, while the slice-specific summaries support the ablations and the failure-case interpretation.

IV. Results

Across the aggregated evaluation slices, the same contract x runtime ordering recurs: P0/R0 is the weakest condition, P0/R1 recovers, and the explicit-contract conditions remain strong.

A. Main Contract x Runtime Effect

The main effect is a stable qualitative ordering across navigation easy, tabletop manipulation easy, harder navigation, and harder manipulation. In each slice, P0/R0 is the weakest cell. Pairing the same under-specified contract with R1 changes the outcome: P0/R1 succeeds where P0/R0 fails, while P1/R0, P1/R1, P2/R0, and P2/R1 remain strong. The same ordering holds across both task families and persists as tasks become harder.

Table II records the same retained cohorts in a reviewer-facing layout that keeps success, invalid-action exposure, and retries readable without forcing planner/tool counts into every cell. Figure 2 makes the same distinction visually: the recovered `P0/R1` cells are separated from the clean `P1/P2` cells, so the repeated 1.0 success values remain interpretable at manuscript scale.

The same comparison separates the roles of contract design and runtime validation. Invalid-action failures are concentrated in the P0 cells, showing that loose task-level verbs are brittle at the planner/executor boundary. Once the planner is forced into a typed tool namespace, the dominant failure mode disappears in the covered fixed-R0 slice. Within the successful structured conditions, P2 matches P1 on success while showing lower planner/tool overhead across the retained `P1` versus `P2` comparisons. Table III isolates that workload comparison so the manuscript can report it without overloading the outcome table. Figure 3 compresses those retained comparisons into one workload mark per row because planner and tool counts are equal within the plotted `P1/P2` pairs. We interpret that pattern as a workload association within already-strong contracts, not as evidence of a new success regime.

TABLE I

FROZEN EVALUATION MATRIX USED BY THE MANUSCRIPT. THE MAIN FACTORIAL ROWS CARRY THE RETAINED ORDERING COMPARISON, WHILE THE TWO COMPACT ABLATIONS ISOLATE CONTRACT AND RUNTIME EFFECTS WITHOUT ADDING NEW EXPERIMENTS.

Block	Family	Slice	Varying axes	Tasks	Runs/cell	Total	Purpose
Main factorial	Navigation	Easy	P0/P1/P2 x R0/R1	11	11	66	Main ordering
Main factorial	Manipulation	Easy	P0/P1/P2 x R0/R1	3	3	18	Main ordering
Main factorial	Navigation	Harder	P0/P1/P2 x R0/R1	4	4	24	Harder robustness
Main factorial	Manipulation	Harder	P0/P1/P2 x R0/R1	3	3	18	Harder robustness
Action-interface ablation	Mixed	Easy shared subset	P0/P1/P2 at fixed R0	4	4	12	Contract mechanism
Runtime-only ablation	Mixed	Recoverable probes	R0/R1 at fixed P1	4	4	8	Runtime recovery

	P0		P1		P2	
Cohort	R0	R1	R0	R1	R0	R1
Navigation Easy	**0/11**	**11/11**	**11/11**	**11/11**	**11/11**	**11/11**
11 runs/cell	fail	recovered	clean	clean	clean	clean
Manipulation Easy	**0/3**	**3/3**	**3/3**	**3/3**	**3/3**	**3/3**
3 runs/cell	fail	recovered	clean	clean	clean	clean
Navigation Harder	**0/4**	**4/4**	**4/4**	**4/4**	**4/4**	**4/4**
4 runs/cell	fail	recovered	clean	clean	clean	clean
Manipulation Harder	**0/3**	**3/3**	**3/3**	**3/3**	**3/3**	**3/3**
3 runs/cell	fail	recovered	clean	clean	clean	clean

Cell text shows successful runs / runs. The harder rows preserve the same ordering as the easy rows; workload growth is summarized separately in Fig. 3 and Table III.

Fig. 2. Main outcome ordering across the retained navigation and manipulation cohorts. The under-specified direct-action cell `P0/R0` is consistently weakest, `P0/R1` recovers, and the typed-contract cells remain successful in the covered slices.

TABLE II

OUTCOME SUMMARY ACROSS THE RETAINED COHORTS. EACH ENTRY KEEPS SUCCESS, INVALID/RUN, AND RETRY/RUN TOGETHER SO THE ARCHIVAL TABLE STILL DISTINGUISHES RECOVERED `P0/R1` CELLS FROM CLEAN STRUCTURED CELLS.

Cell	**Navigation Easy**	**Manipulation Easy**	**Navigation Harder**	**Manipulation Harder**
P0/R0	0/11; 1.0; 0.0	0/3; 1.0; 0.0	0/4; 1.0; 0.0	0/3; 1.0; 0.0
P0/R1	11/11; 1.0; 1.0	3/3; 1.0; 1.0	4/4; 1.0; 1.0	3/3; 1.0; 1.0
P1/R0	11/11; 0.0; 0.0	3/3; 0.0; 0.0	4/4; 0.0; 0.0	3/3; 0.0; 0.0
P1/R1	11/11; 0.0; 0.0	3/3; 0.0; 0.0	4/4; 0.0; 0.0	3/3; 0.0; 0.0
P2/R0	11/11; 0.0; 0.0	3/3; 0.0; 0.0	4/4; 0.0; 0.0	3/3; 0.0; 0.0
P2/R1	11/11; 0.0; 0.0	3/3; 0.0; 0.0	4/4; 0.0; 0.0	3/3; 0.0; 0.0

Cell entries are *success; invalid/run; retry/run*.

TABLE III

PLANNER/TOOL WORKLOAD ACROSS THE RETAINED COHORTS. FIGURE 3 COMPRESSES THE DUPLICATED PLOTTED PAIRS FOR READABILITY; THIS TABLE PRESERVES THE FULL PER-CELL BREAKDOWN.

Cell	**Navigation Easy**	**Manipulation Easy**	**Navigation Harder**	**Manipulation Harder**
P0/R0	1.0/0.0	1.0/0.0	1.0/0.0	1.0/0.0
P0/R1	3.82/2.82	9.0/8.0	6.75/5.75	10.67/9.67
P1/R0	4.82/4.82	10.0/10.0	7.75/7.75	11.67/11.67
P1/R1	4.82/4.82	10.0/10.0	7.75/7.75	11.67/11.67
P2/R0	3.82/3.82	8.0/8.0	6.75/6.75	9.67/9.67
P2/R1	3.82/3.82	8.0/8.0	6.75/6.75	9.67/9.67

Cell entries are *planner calls/tool calls per run*.

B. Action-Interface and Runtime-Only Ablations

The targeted ablations use different easy-task subsets: the runtime-only slice deliberately includes recoverable invalid-first-action probes under fixed P1, whereas the action-interface ablation uses clean easy tasks under fixed R0. Together they isolate the two design axes within the same execution-design framing.

With the contract fixed at P1, moving from R0 to R1

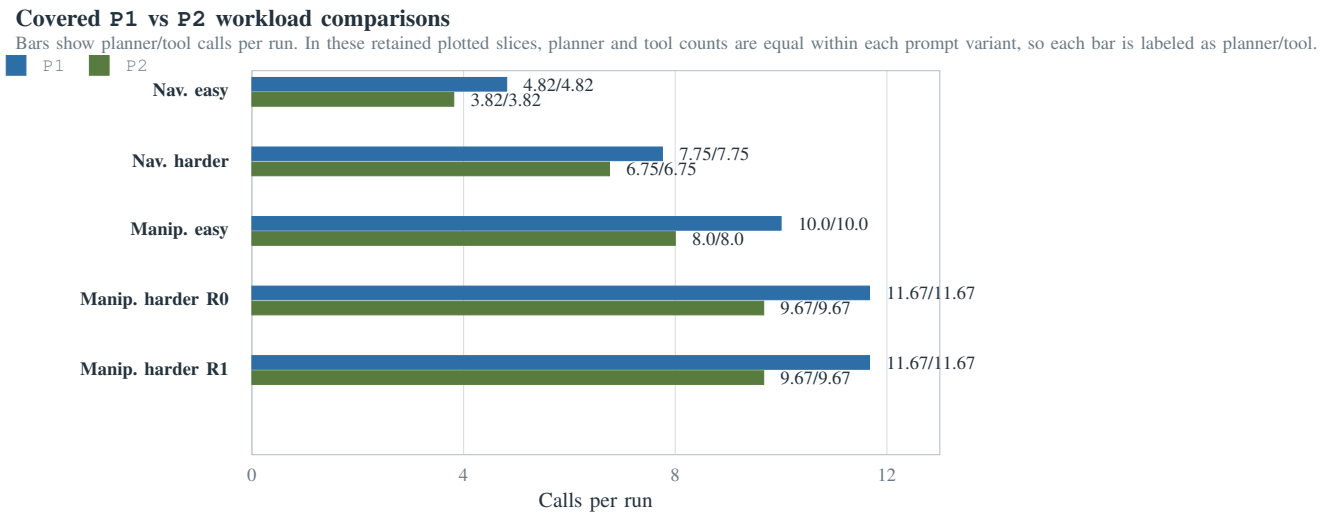

Fig. 3. Workload comparison for the covered `P1` versus `P2` slices. The retained navigation and easy-manipulation cohorts contribute one row each because the `P1`/`P2` workload is identical under `R0` and `R1` there; the harder manipulation slice contributes separate `R0` and `R1` rows. The comparison is descriptive only: in these covered slices, `P2` matches the success profile of `P1` while using fewer planner/tool calls.

raises success from 2/4 to 4/4 across two navigation tasks and two tabletop manipulation tasks. The mechanism is recovery, not suppression: invalid-action runs remain 2 under both runtimes, but successful invalid-action recoveries rise from 0 to 2 once validation and one retry are enabled. The same direction appears in each family separately, with navigation and manipulation each moving from 0.5 to 1.0 success in this slice.

With runtime fixed at R0, P0/R0 fails on all 4/4 evaluated tasks and accumulates four total invalid actions, whereas P1/R0 and P2/R0 succeed on all 4/4 tasks and reduce invalid actions to zero. This establishes an independent contract effect: once the planner must emit dispatchable tool calls, the invalid-action failures that dominate the under-specified interface disappear in this slice. The same comparison also shows the narrower P2 effect: P1/R0 and P2/R0 have the same success profile, but average planner and tool calls each fall from roughly 7.5 to 6.0.

Figure 4 makes both mechanisms visible in one place: the fixed-R0 panel shows typed contracts eliminating invalid actions, while the fixed-P1 runtime panel shows R1 converting the invalid half of the runtime-only slice from failure into recovery, with retries rising from 0.0 to 0.5 per run.

The failure traces explain what goes wrong under P0. In navigation, failing P0 runs repeatedly emit the unsupported tool name `move_to_goal`; in tabletop manipulation, they emit `move_object`. Both are semantically plausible task verbs, but the executor accepts `navigate_to` and `scripted_pick_place_step` instead. R1 helps because the runtime catches the unknown tool before dispatch, returns a validation error, and allows a corrected retry. The runtime does not erase the contract violation; it turns a terminating handoff failure into a recoverable one. Figure 1 includes compact navigation and manipulation traces for exactly that non-dispatchable-to-corrected transition.

A. Fixed-`R0` invalid actions per run

Family	P0	P1	P2
Navigation fixed R0	1.0	0.0	0.0
Manipulatio fixed R0	1.0	0.0	0.0

Typed contracts remove the invalid-action mode in the fixed-`R0` action-interface slice.

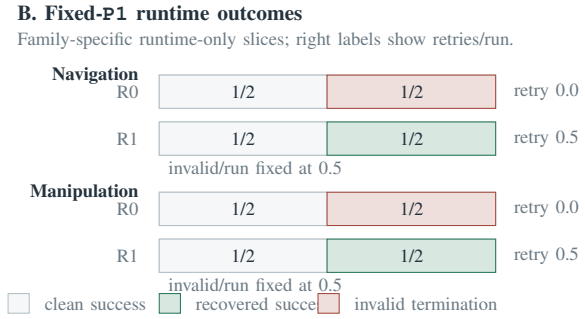

Fig. 4. Mechanism view. Top: under fixed `R0`, `P0` produces one invalid action per run in both families, while `P1`/`P2` reduce that to zero. Bottom: under fixed `P1`, `R1` turns the invalid half of the runtime-only slice into recovered success in both families, with retries rising from 0.0 to 0.5 per run.

C. *Harder-Task Robustness*

The harder-task slices increase planner/tool overhead without changing the condition ordering. In harder navigation, the easy-slice ranking is preserved, while successful cells require roughly three additional planner or tool calls per run. In harder tabletop manipulation, P0/R0 fails on all 3/3 tasks, P0/R1 recovers all 3/3 with one retry per task, and P1/P2 remain fully successful under both runtimes.

The P1 versus P2 workload relation also survives the harder manipulation setting. P2 remains about two planner calls and two tool calls per run below P1 under both runtimes

while maintaining the same success profile. The harder slices therefore look like cost amplification rather than rank reversal.

V. Discussion and Limitations

A. Discussion

The clearest lesson is that planner-to-executor contract design is a systems lever, not a presentation detail. Under the under-specified direct-action contract, the planner often knows the task family but still speaks in a vocabulary the executor cannot dispatch. Explicit contracts move that ambiguity out of ad hoc interpretation and into a declared interface.

Runtime validation plays a complementary role. It does not replace a well-specified contract, but it can intercept a bad first emission, surface a concrete error, and recover the run with minimal additional machinery. That pattern matters for embodied execution stacks that preserve planner-to-executor separation, including middleware-style action interfaces and guarded execution policies.

At a higher level, the studied boundary maps cleanly onto execution architectures already familiar in robotics. The contract specifies the form of a dispatchable request in much the same way that ROS 2 actions expose explicit goal, feedback, and result semantics, while runtime validation decides whether a malformed request should be rejected before execution [12]–[14]. The same separation appears in behavior-tree and TAMP systems: contract checks act like precondition guards on callable nodes or operators, and runtime validation acts like an execution guard that determines whether a proposed action can continue to the executor interface [15]–[18], [20].

The P2 pattern suggests a second, narrower lesson. In the covered slices, a brief planner-side self-check leaves the success profile unchanged while reducing planner/tool overhead. The bounded interpretation is not that self-checks are universally best, but that contract design and runtime design remain separable levers once the callable interface is explicit.

B. Limitations

The conclusions are bounded to a controlled Isaac Sim study. The evidence is simulator-based, covers two task families, and is limited to the current easy and harder slices rather than a broad execution benchmark.

The action-interface ablation and runtime-only ablation are compact mechanism checks rather than broad sweeps, and backend coverage differs across slices. The runtime-only ablation is especially specific: it is designed around recoverable invalid-first-action probes, so its strongest claim is about lightweight repair at that boundary.

These limits still allow a concrete systems conclusion. In the tested setting, planner-to-executor contracts and lightweight runtime safeguards materially change invalid actions, recovery, and planner/tool overhead. The next empirical extension should preserve the same explicit boundary while widening scenes, backends, and execution stacks.

VI. Conclusion

This paper studies how planner-to-executor contract design and runtime validation shape embodied-agent execution in Isaac Sim. Across navigation and tabletop manipulation tasks, the same qualitative ordering remains stable: P0/R0 is weakest, R1 recovers fragile invalid-first-action cases, and the explicit-contract conditions P1 and P2 remain strong across the covered slices.

The failure analysis shows that the dominant errors are contract violations, not generic task collapse. Explicit contracts prevent those failures at the source, runtime validation repairs some that remain, and a brief self-check can reduce planner/tool overhead within an already strong contract. For embodied-agent execution stacks, contract design and lightweight runtime safeguards belong in the architecture, not as afterthoughts.

References

[1] D. Driess, F. Xia, M. S. M. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Tompson, Q. Vuong, T. Yu, W. Huang, Y. Chebotar, P. Sermanet, D. Duckworth, S. Levine, V. Vanhoucke, K. Hausman, M. Toussaint, K. Greff, A. Zeng, I. Mordatch, and P. Florence, "PaLM-E: An embodied multimodal language model," in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 8469–8488. [Online]. Available: https://proceedings.mlr.press/v202/driess23a.html

[2] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid, Q. Vuong, V. Vanhoucke, H. Tran, R. Soricut, A. Singh, J. Singh, P. Sermanet, P. R. Sanketi, G. Salazar, M. S. Ryoo, K. Reymann, K. Rao, K. Pertsch, I. Mordatch, H. Michalewski, Y. Lu, S. Levine, L. Lee, T.-W. E. Lee, I. Leal, Y. Kuang, D. Kalashnikov, R. Julian, N. J. Joshi, A. Irpan, B. Ichter, J. Hsu, A. Herzog, K. Hausman, K. Gopalakrishnan, C. Fu, P. Florence, C. Finn, K. A. Dubey, D. Driess, T. Ding, K. M. Choromanski, X. Chen, Y. Chebotar, J. Carbajal, N. Brown, A. Brohan, M. G. Arenas, and K. Han, "RT-2: Vision-language-action models transfer web knowledge to robotic control," in *Proceedings of The 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229. PMLR, 2023, pp. 2165–2183. [Online]. Available: https://proceedings.mlr.press/v229/zitkovich23a.html

[3] R. Yang, H. Chen, J. Zhang, M. Zhao, C. Qian, K. Wang, Q. Wang, T. V. Koripella, M. Movahedi, M. Li, H. Ji, H. Zhang, and T. Zhang, "EmbodiedBench: Comprehensive benchmarking multi-modal large language models for vision-driven embodied agents," in *Proceedings of the 42nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 267. PMLR, 2025, pp. 70 576–70 631. [Online]. Available: https://proceedings.mlr.press/v267/yang25f.html

[4] X. Lu, Z. Chen, X. Hu, Y. Zhou, W. Zhang, D. Liu, L. Sheng, and J. Shao, "IS-Bench: Evaluating interactive safety of VLM-driven embodied agents in daily household tasks," arXiv preprint arXiv:2506.16402, 2025. [Online]. Available: https://arxiv.org/abs/2506.16402

[5] N. Kachaev, A. Spiridonov, A. Gorodetsky, K. Muravyev, N. Oskolkov, A. Narendra, V. Shakhuro, D. Makarov, A. I. Panov, P. Fedotova, and A. K. Kovalev, "Mind and motion aligned: A joint evaluation IsaacSim benchmark for task planning and low-level policies in mobile manipulation," arXiv preprint arXiv:2508.15663, 2025. [Online]. Available: https://arxiv.org/abs/2508.15663

[6] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, P. Sermanet, N. Brown, T. Jackson, L. Luu, S. Levine, K. Hausman, and B. Ichter, "Inner monologue: Embodied reasoning through planning with language models," in *Proceedings of The 6th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 205. PMLR, 2023, pp. 1769–1782. [Online]. Available: https://proceedings.mlr.press/v205/huang23c.html

[7] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei, "VoxPoser: Composable 3D value maps for robotic manipulation with language models," in *Proceedings of The 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229. PMLR, 2023, pp. 540–562. [Online]. Available: https://proceedings.mlr.press/v229/huang23b.html

[8] P. Liu, Y. Orru, J. Vakil, C. Paxton, N. M. M. Shafiullah, and L. Pinto, "OK-Robot: What really matters in integrating open-knowledge models for robotics," arXiv preprint arXiv:2401.12202, 2024. [Online]. Available: https://arxiv.org/abs/2401.12202

[9] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, A. Herzog, D. Ho, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, E. Jang, R. J. Ruano, K. Jeffrey, S. Jesmonth, N. J. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, K.-H. Lee, S. Levine, Y. Lu, L. Luu, C. Parada, P. Pastor, J. Quiambao, K. Rao, J. Rettinghouse, D. Reyes, P. Sermanet, N. Sievers, C. Tan, A. Toshev, V. Vanhoucke, F. Xia, T. Xiao, P. Xu, S. Xu, M. Yan, and A. Zeng, "Do as I can, not as I say: Grounding language in robotic affordances," in *Proceedings of The 6th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 205. PMLR, 2023, pp. 287–318. [Online]. Available: https://proceedings.mlr.press/v205/ichter23a.html

[10] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, "Code as policies: Language model programs for embodied control," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 9493–9500. [Online]. Available: https://doi.org/10.1109/ICRA48891.2023.10160591

[11] I. Singh, V. Blukis, A. Mousavian, A. Goyal, D. Xu, J. Tremblay, D. Fox, J. Thomason, and A. Garg, "Progprompt: Generating situated robot task plans using large language models," *Autonomous Robots*, vol. 47, pp. 999–1012, 2023. [Online]. Available: https://doi.org/10.1007/s10514-023-10135-3

[12] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, 2022. [Online]. Available: https://doi.org/10.1126/scirobotics.abm6074

[13] G. Biggs, J. Perron, and S. Loretz, "ROS 2 actions," ROS 2 design article, 2019, last modified 2020-05. [Online]. Available: https://design.ros2.org/articles/actions.html

[14] F. Martin, J. G. Clavero, V. Matellan, and F. J. Rodriguez, "PlanSys2: A planning system framework for ROS2," in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2021, pp. 9742–9749. [Online]. Available: https://doi.org/10.1109/IROS51168.2021.9636544

[15] M. Colledanchise and P. Ögren, *Behavior Trees in Robotics and AI*. CRC Press, 2018. [Online]. Available: https://doi.org/10.1201/9780429489105

[16] BehaviorTree.CPP Documentation, "BehaviorTree.CPP docs," Official documentation, version 4.8; accessed 2026-03-18. [Online]. Available: https://www.behaviortree.dev/docs/intro/

[17] C. R. Garrett, T. Lozano-Pérez, and L. P. Kaelbling, "PDDLStream: Integrating symbolic planners and blackbox samplers via optimistic adaptive planning," *Proceedings of the International Conference on Automated Planning and Scheduling*, vol. 30, pp. 440–448, 2020. [Online]. Available: https://doi.org/10.1609/ICAPS.V30I1.6739

[18] C. R. Garrett, R. Chitnis, R. Holladay, B. Kim, T. Silver, L. P. Kaelbling, and T. Lozano-Pérez, "Integrated task and motion planning," *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 4, no. 1, pp. 265–293, 2021. [Online]. Available: https://doi.org/10.1146/annurev-control-091420-084139

[19] NVIDIA, "Isaac Sim documentation," Official documentation, accessed 2026-03-17. [Online]. Available: https://docs.isaacsim.omniverse.nvidia.com/latest/index.html

[20] BehaviorTree.CPP Documentation, "Pre and post conditions," Official documentation, version 4.0.2; accessed 2026-03-18. [Online]. Available: https://www.behaviortree.dev/docs/4.0.2/tutorial-advanced/pre_post_conditions/